Emploid System Architecture & Schema

1. High-Level Architecture

Emploid is a Next.js application that integrates with Supabase (PostgreSQL) for database, authentication, and security, and uses a Python-based pipeline for scraping and data ingestion.

```
graph TD
    Scraper[Python Scrapers] -->|JSON stream via stdin| IngestScript[scripts/ingest.py]
    IngestScript -->|SQL Write via Client API| Supabase[(Supabase Database)]
    NextJS[Next.js App Server] -->|SQL Read/Write| Supabase
    Client[Web Browser] -->|SPA Navigation / UI Interaction| NextJS
    Supabase -->|Triggers| RecalcStats[Update Company Stats]
```

---

2. Frontend Infrastructure

Although Emploid is structured as a Next.js app, it functions as a highly optimized, client-driven Single Page Application (SPA):

- **Core HTML Delivery:** The primary router `app/page.tsx` reads the static file `public/index.html` on the server and injects its body contents into the page using React's `dangerouslySetInnerHTML`.
- **Global Stylesheets:** The Next.js Root Layout (`app/layout.tsx`) binds three stylesheets:
- `style.css` Core utility classes and custom component definitions.
- `colors_and_type.css` Color tokens, typography, and variable overrides.
- `tracker.css` Styles dedicated to the application tracking system.
- **Client Router (`main.js`):** Intercepts internal link clicks, updates the URL history, and modifies the `data-page` attribute on the HTML `<body>` element. Section elements inside `index.html` are styled to display or hide based on the active `data-page` value.
- **Application Tracker (`tracker-app.jsx`):** A React/JSX sub-application loaded dynamically in the tracker tab. It fetches saved jobs, renders stage columns, and handles interactive dragging and progress charts.

---

3. Database Schema

Emploid runs on a PostgreSQL database hosted by Supabase. Below are the key tables, their purposes, and schemas.

Companies Table

Stores details of the companies posting job listings.

```
CREATE TABLE public.companies (
  id uuid DEFAULT gen_random_uuid() PRIMARY KEY,
  name text NOT NULL,
  slug text UNIQUE NOT NULL, -- URL-friendly identifier
  logo_url text,
  industry text,
  size_range text CHECK (size_range IN ('1-50', '51-200', '201-500', '501-1000', '1001-5000',
'5000+')),
  employee_count integer,
  founded_year integer,
  hq_location text,
  website text,
  careers_page_url text,
  glassdoor_rating decimal(2,1),
  glassdoor_url text,
  avg_ghost_score decimal(4,1), -- Aggregated trust average
  total_active_listings integer DEFAULT 0,
  hiring_velocity_90d integer DEFAULT 0,
  last_layoff_date date,
  last_layoff_count integer,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);
```

Jobs Table

Stores job openings, their sources, URLs, metadata, and calculated Trust Scores.

```
CREATE TABLE public.jobs (
  id uuid DEFAULT gen_random_uuid() PRIMARY KEY,
  title text NOT NULL,
  company_id uuid REFERENCES public.companies(id) ON DELETE CASCADE,
  location text,
  remote_type text CHECK (remote_type IN ('remote', 'hybrid', 'onsite')),
  salary_min integer,
  salary_max integer,
  salary_is_estimate boolean DEFAULT false,
  description text,
     source text CHECK (source IN ('linkedin', 'indeed', 'handshake', 'glassdoor',
'ziprecruiter', 'company_direct')),
  source_url text, -- Unique link to job board posting
  apply_url text NOT NULL, -- Direct application link (typically careers page or ATS)
    experience_level text CHECK (experience_level IN ('intern', 'entry', 'mid', 'senior',
'executive')),
  job_type text CHECK (job_type IN ('full-time', 'part-time', 'contract', 'internship')),
  ghost_score integer CHECK (ghost_score BETWEEN 0 AND 100),
  ghost_factors jsonb, -- Structured breakdown of score factors
  ghost_label text CHECK (ghost_label IN ('Verified', 'Uncertain', 'Likely Ghost')),
  is_active boolean DEFAULT true,
  posted_at timestamptz,
  first_seen_at timestamptz DEFAULT now(),
  last_seen_at timestamptz DEFAULT now(),
```

```
  repost_count integer DEFAULT 0,
  search_vector tsvector GENERATED ALWAYS AS (to_tsvector('english', coalesce(title, '') || '
' || coalesce(description, ''))) STORED,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);
```

Recruiters & Job Recruiters Table

Connects job postings to the specific contact details of recruiters or hiring managers.

```
CREATE TABLE public.recruiters (
  id uuid DEFAULT gen_random_uuid() PRIMARY KEY,
  company_id uuid REFERENCES public.companies(id) ON DELETE CASCADE,
  name text NOT NULL,
  title text,
  linkedin_url text,
  email text,
  source text,
  created_at timestamptz DEFAULT now()
);

CREATE TABLE public.job_recruiters (
  job_id uuid REFERENCES public.jobs(id) ON DELETE CASCADE,
  recruiter_id uuid REFERENCES public.recruiters(id) ON DELETE CASCADE,
  PRIMARY KEY (job_id, recruiter_id)
);
```

Users Table

Stores local profiles synced with Supabase Auth users.

```
CREATE TABLE public.users (
  id uuid PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email text,
  name text,
  is_pro boolean DEFAULT false,
  stripe_customer_id text,
  preferred_titles text[],
  preferred_locations text[],
  preferred_salary_min integer,
  experience_level text,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);
```

Saved Jobs (Tracker Table)

Links authenticated users to jobs they have saved or applied to, serving as the backend for the Application Tracker.

```
CREATE TABLE public.saved_jobs (
  id uuid DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id uuid REFERENCES public.users(id) ON DELETE CASCADE,
  job_id uuid REFERENCES public.jobs(id) ON DELETE CASCADE,
   status text DEFAULT 'saved' CHECK (status IN ('saved', 'applied', 'interview', 'offer',
```

```
'rejected')),
  notes text,
  applied_at timestamptz,
  created_at timestamptz DEFAULT now(),
  UNIQUE(user_id, job_id)
);
```

---

4. Row Level Security (RLS) Policies

Supabase restricts access to database records based on user authentication:

- **Companies, Jobs, Recruiters:** Publicly readable by all users (`FOR SELECT USING (true)`).
- **Recruiter Contact Details (PII):** Restrained to authenticated users to prevent scrapers from harvesting recruiter emails/LinkedIn pages.
- **Saved Jobs & Job Alerts:** Owned by individual users; restricted so users can only perform select, insert, update, or delete operations on rows matching their own `user_id` (`auth.uid() = user_id`).

---

5. Data Ingestion Pipeline

Jobs are updated in the database through a automated pipeline: 1. **Scraping:** Custom Python scrapers fetch active listings from target sites (e.g., Greenhouse API, LinkedIn, Indeed) and output normalized JSON objects. 2. **Streaming:** JSON items are piped line-by-line via standard input (stdin) to `scripts/ingest.py`. 3. **Company Resolution:** For each job, the script checks if the company exists by generating a clean URL-friendly slug using a custom `slugify` regex. If it does not exist, the script upserts a new record to the `companies` table. 4. **Deduplication:** The script attempts to find matching jobs in the database by checking:

- First, the unique `source_url` field (unique index prevents duplicates).
- Second, a composite key match of `title` + `company_id` + `location` + `source`.

5. **Upsert Operation:**

- **If the job already exists:** The script updates `last_seen_at` to the current timestamp and overwrites fields that are present in the new payload.
- **If the job is new:** The script inserts the job as active, initializing `first_seen_at` and `last_seen_at` to the current time.

6. **Trigger-Based Aggregations:** After a job is inserted, updated, or deleted, a database trigger `update_company_stats_trg` automatically recalculates:

- `total_active_listings` (count of jobs with `is_active = true` for that company).
- `avg_ghost_score` (average trust score of active company listings).